

Vegetation Health Classification for Low-Data and Edge Hardware Deployment

Yifei Cheng and Haoyu Li¹ 

College of Computing, Grand Valley State University, Allendale, MI, USA

Abstract. Drone-based vegetation health monitoring typically relies on deep learning or kernel-based methods, requiring large annotated datasets, GPU hardware, and long retraining cycles when new data is added. These constraints make them difficult to work with for consumer drones and low-power field hardware used by independent practitioners for environmental monitoring. This study aims to create a lightweight, interpretable pipeline to classify vegetation imagery for drone monitoring. Our method is based on 2D hue-saturation histograms compared against class archetypes using Earth Mover’s Distance or a custom Quadratic Form Distance (QFD) variant optimized in C++ for memory-constrained hardware. To test the effectiveness of these pipelines, we evaluated against 3 other baselines: a Siamese network, Random Forest, and SVM, across four data regimes. The test results reveal our pipelines achieve competitive or superior Top-1 accuracy under limited labeled-data scenarios while remaining fast, interpretable, and easily updatable through simple reaggregation. The QFD variant deployed on an ESP32 achieves 82.8% accuracy at just $\sim 1.01\%$ heap utilization (45,136 bytes), demonstrating that histogram-based methods can match or exceed neural few-shot approaches in limited labeled-data scenarios and low compute power settings.

Keywords: Low-Data Learning · Data-Efficient Classification · Color Histogram Classification

1 Introduction

Drone-captured color imagery is popular for vegetation health monitoring when infrared imaging is unavailable or cost-prohibitive [9, 10]. However, existing classification approaches have distinct drawbacks. Supervised deep learning and SVM methods require thousands of annotated images per class and complete retraining for new data, making them infeasible for independent field operators [6]. Few-shot learning mitigates data scarcity but relies on deep feature extractors like ResNet [5] and EfficientNet [7] that strain onboard drone processors and microcontrollers like the ESP32 [3], while still requiring retraining [8]. In addition, many existing approaches function as black boxes, offering little interpretability for manual tuning or auditing [4]. Ultimately, these gaps in data efficiency, interpretability, and compute power restrict deployment in low-resource conservation efforts.

2 New Approach

The proposed pipelines utilize a 2D Hue-Saturation histogram classification method [4]. Input RGB images are converted to HSV, discarding pixels with Value > 240 or < 20 to eliminate glare and shadows. The conversion maps RGB values to HSV space, calculating Hue (H) as an angular position, Saturation (S) as color purity, and Value (V) as maximum intensity. The 2D histogram is built by mapping continuous H and S coordinates into discrete spatial grid bins and accumulating eligible pixel frequencies. A normalized 2D histogram is built using 15 hue bins (24° each) and 8 saturation bins (12.5% each) by accumulating pixel frequencies.

The pipeline creates three reference histograms per class, using Earth Mover’s Distance (EMD) or Quadratic Form Distance (QFD) to identify the closest class. Both metrics apply a polar saturation weighting scheme that penalizes hue differences at higher saturations to handle the perceptual ambiguity of low-saturation pixels. EMD evaluates cross-bin similarity via linear programming: $EMD(P, Q) = \frac{\min_{F \in \mathcal{F}} \sum_i \sum_j f_{ij} d_{ij}}{\sum_i \sum_j f_{ij}}$, where $F = [f_{ij}]$ is the optimal flow between bins of histograms P and Q , and d_{ij} is the perceptually weighted ground distance. QFD bypasses iterative optimization using matrix operations: $QFD(P, Q) = \sqrt{(P - Q)^T A (P - Q)}$, where A is a similarity matrix with entries $a_{ij} = 1 - (d_{ij}/d_{\max})$.

The 15×8 resolution was selected iteratively: a coarser 8×4 configuration biased classifications toward the mixed class, whereas the finer resolution captured tighter boundaries. EMD was replaced by QFD because its linear programming optimization exceeded ESP32 memory constraints [3]. Incorporating Local Binary Pattern texture features yielded only a $\sim 0.71\%$ accuracy gain at an unacceptable 100ms inference cost, while a naive RGB-averaging baseline performed near chance level (50.84% Top-1 at $n = 5$).

3 Comparative Study Methodology

Six systems are evaluated across four identical data conditions inside Google Colab. Custom-Avg, Custom-Med, QFD (our three proposed variants), a Siamese network (Few-Shot neural baseline) [8], a Random Forest Model [2], and an SVM [6]. These three baselines are the three dominant paradigms in the literature: kernel-based supervised learning (SVM), ensemble methods (Random Forest), and few-shot neural approaches (Siamese Network). All systems are tested at $n = 5, 10$, and 25 reference images per class and on the full dataset (2,172 drone images across three classes: dry, mixed, and dense vegetation, sourced from 28 South American locations from OpenAerialMap [1])

Evaluation metrics include Top-1 accuracy, partial accuracy (adjacent class placement), inference time, and training time. Top-1 accuracy is a macro average of the percentage of correct predictions for each class. Due to a lack of a reliable way to separately measure memory footprint during the Google Colab session,

memory footprint during inference was measured on the ESP32 to demonstrate memory efficiency on the QFD variant.

The QFD custom pipeline variant is inference tested on an ESP32-WROVER microcontroller at $n = 5$ to demonstrate feasibility and memory efficiency, using training parameters calculated from a computer. The ESP32-WROVER has 520kb of internal SRAM and an additional 4mb of PSRAM. The ESP32 receives an image stream and reference histograms from the computer. The algorithm is rewritten in C++ and contains a few optimizations to operate on the ESP32.

4 Results

Table 1: Top-1 Macro accuracy (top row) and partial accuracy (bottom row) across data regimes. Google Colab session. $\pm$: Standard Deviation across 20 rounds

	n = 5	n = 10	n = 25	All
EMD Averaging	81.67% $\pm$ 5.93% 99.97%	84.67% $\pm$ 4.83% 100%	85.80% $\pm$ 2.36% 100%	85.06% 100%
EMD Median	79.00% $\pm$ 7.39% 99.86%	81.00% $\pm$ 5.70% 99.87%	83.67% $\pm$ 5.14% 99.97%	83.22% 100%
QFD Variant	81.43% $\pm$ 4.60% 99.97%	82.67% $\pm$ 4.20% 100%	84.34% $\pm$ 2.21% 100.00%	85.29% 100%
Siamese Network	67.33% $\pm$ 9.73% 98.56%	76.33% $\pm$ 5.80% 99.69%	84.33% $\pm$ 5.47% 99.96%	93.25% 100%
Random Forest	79.33% $\pm$ 5.20% 94.00%	85.33% $\pm$ 4.61% 98.00%	89.00% $\pm$ 2.31% 98%	94.67% 100%
SVM	58.33% $\pm$ 11.86% 71.00%	68.59% $\pm$ 8.42% 85.00%	77.33% $\pm$ 4.20% 95%	90.84% 100%

In terms of accuracy, across the low-data regimes, our three custom pipeline variants (EMD Average, EMD Median, QFD Variant) consistently achieve competitive or superior Top-1 accuracy to the other baselines. Although EMD Median achieved a lower Top-1 accuracy compared to EMD Average on all data scenarios, there is far less of a class imbalance against the mixed class, suggesting that EMD Median is more robust towards identifying intermediary classes.

SVM struggles most severely at low n , confirming that kernel-based classifiers require substantially more data to generalize. The Random Forest Model and Siamese Network emerge as strong competitors at higher n values, eventually achieving the highest Top-1 accuracies in the full-data scenario, though at the cost of significantly weaker partial accuracy compared to our custom pipelines at low n .

QFD Variant achieves the fastest inference ($<0.4\text{ms}$) across all data regimes, and Systems EMD Average and EMD Median incur higher inference costs due to EMD’s iterative optimization. For all non-neural systems, training times remain

Table 2: Training time per round in seconds (top row) and inference time per image in milliseconds (bottom row) from Google Colab session.

	n = 5	n = 10	n = 25	All
EMD Averaging	0.805 ± 0.126	1.415 ± 0.022	3.771 ± 0.496	103.126 ± 4.649
	14.236 ± 11.626	15.437 ± 12.390	15.387 ± 12.849	16.906 ± 13.202
EMD Median	0.731 ± 0.107	1.458 ± 0.221	3.707 ± 0.489	$108.456s \pm 3.177s$
	8.582 ± 6.939	7.644 ± 6.083	6.951 ± 5.428	6.779 ± 5.011
QFD Variant	0.735 ± 0.114	1.539 ± 0.256	3.667 ± 0.495	83.517 ± 3.642
	0.315 ± 0.074	0.316 ± 0.084	0.314 ± 0.086	0.328 ± 0.090
Siamese Network	173.850 ± 4.372	347.450 ± 4.012	892.863 ± 7.684	20014.526
	19.896 ± 4.139	19.896 ± 4.139	20.067 ± 4.281	19.984 ± 3.908
Random Forest	1.110 ± 0.201	1.824 ± 0.274	4.035 ± 0.434	67.892 ± 0.620
	16.196 ± 5.132	15.919 ± 4.384	15.990 ± 4.701	16.004 ± 4.226
SVM	0.763 ± 0.174	1.443 ± 0.197	3.480 ± 0.272	91.013 ± 1.457
	1.089 ± 0.351	1.076 ± 0.234	1.092 ± 0.241	1.280 ± 0.338

tightly clustered (under 5 seconds at $n = 25$), enabling practical field retraining. The Siamese Network scales poorly—training surges from 174 seconds at $n = 5$ to over five hours in the full-data scenario, revealing a major structural barrier for deployments requiring regular field updates.

To validate resource-constrained feasibility, the QFD Variant was deployed on an ESP32 [3] at $n = 5$, achieving 82.8% overall accuracy while utilizing just 45,136 bytes of heap memory (1.01% utilization) with a fragmentation rate of 6.41%.

Per image: Total round-trip : avg= 326.09ms min= 325.18ms max= 328.26ms
 On-device infer : avg= 23.117ms min= 22.381ms max= 23.477ms Transfer+overhead:
 avg= 302.97ms min= 301.81ms max= 305.36ms

5 Conclusion

These results suggest that lightweight histogram-based methods are competitive with or superior to neural few-shot approaches in low-data regimes, while also being faster during inference and deployable on microcontrollers and low-end IoT devices without GPU hardware. The performance gap at full data reflects a known ceiling of color histogram representations and is expected. The reaggregation-based update scheme further removes one of the most significant operational roadblocks of field deployment, eliminating the requirement to re-train when new data is added.

The successful ESP32 deployment validates that on-device inference is practically achievable and accurately quantifies real-world memory usage. Future work should examine whether incorporating lightweight texture features or spatial structure could narrow the full-data accuracy gap without sacrificing edge deployability.

References

1. Openaerialmap. <https://openaerialmap.org>, accessed: 2026
2. Azmi, F., Gibran, M.K., Saleh, A.: Herbal plant image retrieval using hsv color histogram and random forest algorithm. *Journal of Computer Science, Information Technology and Telecommunication Engineering* **6**(2), 1019–1027 (2025)
3. Espressif Systems: ESP32-WROVER-E & ESP32-WROVER-IE Datasheet (2024), version 1.4
4. Hassanein, M., Lari, Z., El-Sheimy, N.: A new vegetation segmentation approach for cropped fields based on threshold detection from hue histograms. *Sensors* **18**(4), 1253 (2018)
5. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. pp. 770–778 (2016)
6. Jintasuttisak, T., Chabplan, P., Issaro, S., Saeung, O., Suwanroj, T.: Accurate segmentation of vegetation in uav desert imagery using hsv-glm features and svm classification. *Journal of Imaging* **12**(1), 9 (2025)
7. Tan, M., Le, Q.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: *International conference on machine learning*. pp. 6105–6114. PMLR (2019)
8. Uzhinskiy, A.V., Ososkov, G.A., Goncharov, P.V., Nechaevskiy, A.V., Smetanin, A.A.: One-shot learning with triplet loss for vegetation classification tasks. *Computer Optics (In Russian)* **45**(4), 608–614 (2021)
9. Wenbo, Y., Peng, G., Honglei, S., Wei, Z., Fuqiang, L.: Identification of grassland vegetation coverage and height based on vegetation index and hsv space. *Journal of Resources and Ecology* **16**(4), 933–945 (2025)
10. Yang, W., Wang, S., Zhao, X., Zhang, J., Feng, J.: Greenness identification based on hsv decision tree. *Information Processing in Agriculture* **2**(3-4), 149–160 (2015)